PROGRAMACIÓN IV

Trabajo Práctico - CRUD Categorías + CRUD Productos

OBJETIVO GENERAL

Desarrollar una aplicación Full Stack conectando un frontend React + TypeScript con un backend FastAPI. El estudiante implementará el CRUD completo de Categorías desde la interfaz de usuario, integrando el backend mediante fetch nativo. Los CRUDs de Producto y ProductoCategoria serán construidos en el backend y demostrados mediante un archivo REST Client.

MARCO TEÓRICO

Concepto	Aplicación en el proyecto
React	Biblioteca para construir interfaces de usuario mediante componentes reutilizables.
TypeScript	Superset tipado de JavaScript para detectar errores en compilación y mejorar la legibilidad.
Vite	Herramienta de build moderna que inicializa proyectos React con TypeScript, reemplazando Create React App.
FastAPI	Framework Python de alto rendimiento para construir APIs REST. Permite definir endpoints, validar datos con Pydantic y exponer documentación automática en /docs.
Pydantic	Librería de validación de datos usada por FastAPI para definir modelos (schemas) con tipado estricto.
fetch nativo	API del navegador para realizar peticiones HTTP. Se usa con async/await para consumir la API REST desde el frontend.
useState	Hook de React para manejar estado local en un componente funcional (ej: lista de categorías, estado del modal).
useEffect	Hook de React para ejecutar efectos secundarios (ej: fetch al montar el componente para cargar datos).
Interface	Construcción de TypeScript que define la forma de un objeto (ej: Categoria, Producto).
Tailwind CSS	Framework de CSS utilitario. Estilos aplicados directamente en el JSX mediante clases predefinidas.
CORS	Mecanismo que permite al frontend (localhost:5173) comunicarse con el backend (localhost:8000) en dominios distintos.

CASO PRÁCTICO

Parte A — Backend con Fast Api

Tomando como base el trabajo práctico de la unidad 6:

a) Crear y activar un entorno virtual Python, crea archivo de dependencias e instalar las dependencias:

```
python -m venv .venv
source .venv/bin/activate   # macOS/Linux
.venv\Scripts\activate      # Windows
pip install -r requirements.txt
```

b) Definir los models y schemas del modulo Catálogo de Productos restantes.
(SVG adjunto a este pdf)

c) Al módulo “Catálogo de productos” agregar campos faltantes (Auditoría)
d) Crear un CRUD siguiendo la arquitectura **Models-Schemas-Service-Router**

Endpoint Necesarios:

Categorías

Método	Endpoint	Body	Response	Query Params
GET	**/categorias/**	**—**	**CategoriaPaginatedResponse**	**offset, limit, nombre, incluir_inactivos**
POST	**/categorias/**	**CategoriaCreate**	**CategoriaRead (201)**	**—**
GET	**/categorias/{id}**	**—**	**CategoriaRead**	**—**
PUT	**/categorias/{id}**	**CategoriaUpdate**	**CategoriaRead**	**—**
DELETE	**/categorias/{id}**	**—**	**204 No Content**	**—**

Productos

Método	Endpoint	Body	Response	Query Params
GET	/productos/	—	ProductoPaginatedResponse	offset, limit, nombre, incluir_inactivos
POST	/productos/	ProductoCreate	ProductoRead (201)	—

GET	/productos/{id}	—	ProductoRead	—
PUT	/productos/{id}	ProductoUpdate	ProductoRead	—
DELETE	/productos/{id}	—	204 No Content	—

Relaciones N:M (solo consulta)

No existen endpoints POST/DELETE para gestionar relaciones. Las asociaciones se crean/actualizan ÚNICAMENTE al crear o editar el producto.

Método	Endpoint	Descripción
GET	/productos/{id}/categorias	Categorías de un producto
GET	/productos/{id}/ingredientes	Ingredientes de un producto
GET	/categorias/{id}/productos	Productos de una categoría

Ingredientes

Método	Endpoint	Body	Response	Query Params
GET	/ingredientes/	—	IngredientePaginatedResponse	offset, limit
POST	/ingredientes/	IngredienteCreate	IngredienteRead (201)	—
GET	/ingredientes/{id}	—	IngredienteRead	—
PUT	/ingredientes/{id}	IngredienteUpdate	IngredienteRead	—
DELETE	/ingredientes/{id}	—	204 No Content	—

Unidades de Medida (Catálogo — solo lectura)

Método	Endpoint	Body	Response
GET	/unidades-medida/	—	UnidadMedidaListResponse
GET	/unidades-medida/{id}	—	UnidadMedidaRead

Health

Método	Endpoint	Response
GET	/health/	200 OK
GET	/health/db	200 OK

Query Parameters (comunes)

Parámetro	Tipo	Default	Descripción
offset	int	0	Desplazamiento para paginación

limit	int	10	Cantidad de resultados por página
nombre	string	—	Filtro por nombre (búsqueda parcial)
incluir_inactivos	bool	false	Incluir registros eliminados (soft delete)

Seed Data — Unidades de Medida

ID	Nombre	Símbolo	Tipo
1	kilogramo	kg	masa
2	gramo	g	masa
3	litro	L	volumen
4	mililitro	mL	volumen
5	pieza	u	unidad
6	docena	doc	unidad
7	metro cuadrado	m²	area

Notas:

- Recuerda agregar CORS para poder consumir tu API desde su frontend
- No debes de borrar nada de la Base de Datos, debes utilizar borrado lógico (En todos los modelos).

```
Recuerda que:
Las tablas Catalog se cargan a partir de un archivo seed.py. No deben de
tener operaciones de escritura (POST/PUT/PATCH) solo deben ser operaciones
de lectura (GET)
```

e) Los endpoints deberán de tener la optimización aprendida en la unidad y respuestas estandarizadas

f) Ejecutar el servidor:

```
python -m fastapi dev main.py
```

📘 Parte B — Frontend con React + TypeScript (MEJORADO)

g) Páginas (Routing)

La aplicación debe estar organizada en páginas separadas.

Crear en src/pages/:

- ***CategoriasPage.tsx***
- ***ProductsPage.tsx***

CategoriasPage

Debe contener:

- *Toda la lógica de categorías:*
  - *useQuery*
  - *useMutation*
  - *manejo del modal*
- *Renderiza:*
  - *CategoriaList*
  - *CategoriaModal*
  - *botón "Nueva categoría"*

ProductsPage

- *Estructura similar a CategoriasPage*
- *Preparada para trabajar con productos*
- *Puede reutilizar lógica (recomendado)*
- *Debe tener:*
  - *listado*
  - *formulario (modal o página)*

h) Navegación

Implementar rutas usando React Router.

Instalar:

pnpm add react-router-dom

Configuración básica

Definir rutas:

- */categorias → CategoriasPage*
- */productos → ProductsPage*

Navbar

Actualizar Navbar.tsx para incluir navegación:

- *Link a **Categorías***
- *Link a **Productos***

i) App.tsx (cambio clave)

*Ahora **App ya no tiene lógica de negocio**.*

Solo debe encargarse de:

- *Definir rutas*
- *Renderizar el Navbar*
- *Mostrar las páginas*

Comportamiento esperado (actualizado)

- *La app inicia en /categorias*
- *Se puede navegar a /productos*
- *Cada página maneja su propia lógica*
- *No hay lógica mezclada en App (orden profesional)*

HISTORIAS DE USUARIO

Las siguientes historias de usuario describen, desde la perspectiva del actor que opera el sistema, el comportamiento esperado de la aplicacion. Cada historia sigue el formato "Como... quiero... para..." y se acompania de sus criterios de aceptacion, que constituyen la condicion verificable de que la historia fue implementada correctamente.

ID	Historia de usuario y criterios de aceptacion	Canal de verificacion

HU-01	Listar categorias Como administrador del catalogo, quiero ver al ingresar a la aplicacion la lista de todas las categorias existentes, para tener una vision inmediata de como esta organizado el inventario. Criterios de aceptacion: • Al montar CategoriasPage se ejecuta useQuery con GET /categorias, que responde 200 con la lista. • Cada categoria se muestra con nombre y descripcion en la pagina. • Si no existen categorias, se muestra un estado vacio comprensible. • Si la peticion falla, se informa el error al usuario con el formato {"detail": "..."}.	Frontend (React)
HU-02	Crear una categoria Como administrador del catalogo, quiero crear una nueva categoria mediante un formulario en un modal, para incorporar al inventario una clasificacion que todavia no existe. Criterios de aceptacion: • Un boton "Nueva categoria" abre el modal con el formulario de alta vacio. • El formulario valida nombre obligatorio con min_length=1 y rechaza solo espacios. • Al guardar se envia POST /categorias, que responde 201 con la categoria creada, o 409 si ya existe el nombre (RN-02). • Tras una respuesta exitosa, la nueva categoria aparece en la lista sin recargar la pagina y el modal se cierra.	Frontend (React)

HU-03	Editar una categoria Como administrador del catalogo, quiero editar una categoria existente desde el mismo modal, para corregir o actualizar su informacion. Criterios de aceptacion: • El boton Editar de la tarjeta abre el modal con los campos precargados con los datos actuales de la categoria. • Al guardar se envia PUT /categorias/{id} (reemplazo total), que responde 200 con la categoria actualizada, 404 si el id no existe, o 409 si hay duplicado de nombre. • Tras la respuesta exitosa, la tarjeta correspondiente refleja los nuevos valores y el modal se cierra. • El modal distingue claramente entre modo alta y modo edicion.	Frontend (React)
HU-04	Eliminar una categoria Como administrador del catalogo, quiero eliminar una categoria que ya no se utiliza, para mantener el inventario limpio y libre de clasificaciones obsoletas. Criterios de aceptacion: • El boton Eliminar dispara DELETE /categorias/{id}, que responde 204 sin cuerpo, 404 si no existe, o 409 si la categoria tiene productos asociados (RN-05). • Tras la respuesta exitosa, la categoria desaparece de la lista de forma inmediata. • Se solicita confirmacion antes de eliminar para evitar borrados accidentales.	Frontend (React)

HU-05	Navegar entre paginas con React Router Como administrador del catalogo, quiero navegar entre las secciones de Categorias y Productos sin recargar la pagina, para trabajar de forma fluida. Criterios de aceptacion: • El Navbar contiene enlaces a /categorias y /productos usando <Link> de React Router. • La navegacion entre rutas no recarga el navegador; solo cambia el contenido de la pagina. • App.tsx no contiene logica de negocio: solo define rutas y renderiza el Navbar. • Cada pagina maneja su propia logica con hooks (useState, useEffect).	Frontend (React)
HU-06	CRUD de Productos desde REST Client Como administrador del catálogo, quiero listar, crear, actualizar y eliminar productos desde el backend, para gestionar la oferta del inventario. Criterios de aceptación: • GET /productos devuelve la colección completa de productos. • POST /productos crea un producto validado con Pydantic (201). El body debe incluir categoria_ids (lista con al menos 1 ID). Opcionalmente puede incluir ingredientes. Si falta categoria_ids o está vacío, rechaza con 422. • PUT /productos/{id} actualiza los datos (200) o devuelve 404. Si se envían categoria_ids o ingredientes, se reemplazan completamente las relaciones existentes. • DELETE /productos/{id} elimina el producto y en cascada sus asociaciones (204) o 404. • Todas las operaciones se demuestran con archivos .http.	Backend (REST Client)

HU-07	Consultar relación N:M Producto-Categoría y Producto-Ingrediente Como administrador del catálogo, quiero consultar las categorías e ingredientes asociados a un producto y los productos de una categoría, para conocer la organización del inventario. Criterios de aceptación: • GET /productos/{id}/categorias devuelve las categorías del producto. • GET /productos/{id}/ingredientes devuelve los ingredientes del producto. • GET /categorias/{id}/productos devuelve los productos de una categoría. • Las relaciones N:M se gestionan ÚNICAMENTE al crear o actualizar un producto (POST /productos y PUT /productos/{id}). NO existen endpoints específicos para asociar/desasociar por separado. • Consultar contra entidad inexistente devuelve 404.	Backend (REST Client)

REGLAS DE NEGOCIO

Las reglas de negocio definen las restricciones que el sistema debe hacer cumplir para mantener la integridad de los datos. Son responsabilidad del backend (capa de validacion con Pydantic y logica de los servicios) y complementan las historias de usuario: una historia describe que quiere lograr el usuario, mientras que una regla de negocio establece que esta permitido y que debe rechazarse.

ID	Regla de negocio	Entidad / Verificacion
RN-01	El nombre de una categoria es obligatorio (min_length=1) y no puede quedar vacio ni contener solo espacios.	Categoria / 422
RN-02	El nombre de una categoria es unico: no pueden existir dos categorias con el mismo nombre (comparacion sin distincion de mayusculas y espacios externos).	Categoria / 409 Conflict
RN-03	La descripcion de una categoria es opcional; se declara con valor por defecto (descripcion: str = "") y se almacena como cadena vacia si se omite.	Categoria / 201 o 200
RN-04	No se puede actualizar ni eliminar una categoria cuyo id no exista.	Categoria / 404 Not Found
RN-05	Una categoria con productos asociados no puede eliminarse hasta desasociar esos productos o eliminarlos.	Categoria / 409 Conflict

RN-06	El nombre de un producto es obligatorio (min_length=1) y no puede quedar vacio ni contener solo espacios. No es unico: pueden coexistir productos con el mismo nombre.	Producto / 422
RN-07	El precio_base de un producto debe ser >= 0 (Field(ge=0)); no se aceptan precios negativos.	Producto / 422
RN-08	El campo disponible es booleano; se declara con valor por defecto True.	Producto / 201
RN-09	El campo imagenes_url es una lista de cadenas; puede estar vacia.	Producto / 422
RN-10	No se puede consultar, actualizar ni eliminar un producto cuyo id no exista.	Producto / 404 Not Found
RN-11	Al crear o actualizar un producto, todas las categorías incluidas en categoria_ids deben existir previamente. Si alguna no existe, la operación se rechaza con 404.	Relacion / 404
RN-12	Si se envían categoria_ids en un PUT /productos/{id}, se reemplazan TODAS las asociaciones existentes del producto. No es necesario desasociar manualmente.	Relacion / 409 Conflict
RN-13	Un producto puede asociarse a varias categorias y una categoria puede agrupar varios productos (cardinalidad N:M).	Relacion / verificable con GET inversos
RN-14	Al eliminar un producto, se eliminan automaticamente todas sus asociaciones con categorias (borrado en cascada).	Relacion / consistencia tras DELETE
RN-15	El id de cada entidad lo asigna el backend de forma autoincremental (entero, desde 1); el cliente nunca lo envia al crear.	Todas / id de solo lectura
RN-16	Los datos se mantienen en memoria durante la ejecucion del servidor; al reiniciar, el catalogo vuelve a su estado inicial.	Todas / comportamiento esperado
RN-17	Toda solicitud con datos invalidos, entidad inexistente o conflicto de estado no modifica el catalogo y devuelve el error correspondiente.	Todas / 404, 409 o 422
RN-18	La aplicacion gestiona la navegacion del lado del cliente con React Router; el servidor no interviene en el enrutamiento interno.	Frontend / comportamiento esperado
RN-19	Al crear un producto, categoria_ids es obligatorio y debe contener al menos un ID de categoría (Field(min_length=1)). Si se omite o está vacío, la operación se rechaza con 422.	Producto / 422

RN-20	Los ingredientes de un producto son opcionales tanto al crear como al actualizar. Si no se envían, el producto se crea sin ingredientes asociados.	Producto / 201
RN-21	Si se envían ingredientes en PUT /productos/{id}, se reemplazan TODOS los ingredientes existentes del producto por los nuevos.	Producto / 200
RN-22	No existen endpoints POST/DELETE específicos para gestionar relaciones N:M. Las asociaciones se gestionan exclusivamente a través de POST /productos y PUT /productos/{id}.	Relación / —

CONCLUSIONES ESPERADAS

Al finalizar el trabajo práctico, el estudiante debe demostrar:

- **Integración Full Stack:** El frontend React consume la API FastAPI mediante fetch nativo. El CRUD de Categorías funciona de extremo a extremo.
- **Modal con useState:** El formulario de alta y edición está dentro de un modal controlado con estado booleano.
- **Hooks aplicados correctamente:** useState y useEffect utilizados en App.tsx. Los demás componentes son funcionales puros sin hooks.
- **Tipado TypeScript:** Interfaces Categoria y Producto definidas. Props tipados en todos los componentes .tsx.
- **Backend REST documentado:** Endpoints de Producto y ProductoCategoria funcionando y demostrados en el archivo .http.
- **CORS configurado:** El backend permite peticiones desde el origen del frontend.

RÚBRICA DE EVALUACIÓN

Criterio	Descripción	Puntos
CRUD Categorías – Frontend	Listar, crear, editar y eliminar categorías desde React usando fetch nativo y useState/useEffect.	30
Modal funcional	Modal que se abre para crear y editar categorías, controlado con useState. Se cierra al guardar o cancelar.	15
CRUD Categorías – Backend	Endpoints FastAPI GET /categorias, POST, PUT/{id}, DELETE/{id} con Pydantic y persistencia en lista/DB.	20

CRUD Productos y relación – REST Client	Endpoints de Productos y ProductoCategoria demostrados con REST Client (.http). No requiere UI.	15
Tipado TypeScript	Interfaces Categoria y Producto tipadas. Props tipados en todos los componentes.	10
Estructura y buenas prácticas	Carpetas components/ y types/. Tailwind CSS aplicado. Sin errores de TS en consola.	10

Modalidad de Entrega

- Eliminar la carpeta node_modules y venv antes de comprimir
- Proyecto Frontend comprimido en un archivo .zip
- Proyecto Backend comprimido en un archivo .zip

RECURSOS ADICIONALES

📚 **Documentación oficial:**

- React: https://react.dev/
- TypeScript: https://www.typescriptlang.org/docs/
- Tailwind CSS: https://tailwindcss.com/docs/
- Vite: https://vitejs.dev/
- FastAPI: https://fastapi.tiangolo.com/